

Практическое занятие 3.3.1

Проектирование архитектуры Big Data-системы и промышленного пайплайна обработки данных

Дисциплина: Системы обработки больших данных

Студент: Трифонов Дмитрий Сергеевич

Группа: 12-25РПм

Вариант: 12

Дата: 14.04.2026

Цель работы

Спроектировать концептуальную архитектуру Big Data-системы и промышленный пайплайн обработки данных для выбранной прикладной задачи.

```
In [38]: import pandas as pd
from IPython.display import display, Markdown
```

```
In [39]: variant_number = 12

variant_tasks = {
    12: 'Спроектировать архитектуру системы анализа заявок в службу технической поддержки ИТ-и
}

display(Markdown(f'### Вариант {variant_number}'))
display(Markdown(variant_tasks[variant_number]))
```

Вариант 12

Спроектировать архитектуру системы анализа заявок в службу технической поддержки ИТ-компаний с автоматической маршрутизацией обращений.

1. Постановка задачи

Формулировка варианта:

Вставьте или уточните текст варианта.

Спроектировать архитектуру системы анализа заявок в службу технической поддержки ИТ-компаний с автоматической маршрутизацией обращений.

Краткое описание предметной области:

Опишите предметную область в 1–2 абзацах.

Служба технической поддержки (Service Desk) ИТ-компаний обрабатывает обращения пользователей, связанные с ошибками в работе программного обеспечения, инфраструктуры и сервисов. Заявки поступают из различных каналов: веб-интерфейс, электронная почта, чат-боты и автоматические системы мониторинга. Обращения могут содержать как структурированные данные (тип проблемы, пользователь, время), так и неструктурированные (текст описания проблемы). В условиях большого потока заявок возникает необходимость автоматизации их

обработки, включая классификацию, определение приоритета и маршрутизацию в соответствующие группы поддержки.

Основная цель системы:

Сформулируйте, какую проблему должна решать проектируемая система.

Разработать систему, которая автоматически анализирует входящие заявки, определяет их категорию и приоритет, и направляет их в соответствующую группу специалистов, сокращая время обработки и повышая качество обслуживания пользователей.

2. Краткие определения

Кратко раскройте термины: **Big Data, data pipeline, источник данных, хранилище данных, вычислительный слой, аналитический слой, offline-аналитика, online-аналитика, оркестрация, Data Engineer, Data Scientist.**

Big Data — это большие объёмы данных, характеризующиеся высокой скоростью поступления, разнообразием и требующие распределённых технологий хранения и обработки.

Data pipeline — это последовательность этапов обработки данных от их получения из источников до формирования конечного результата.

Источник данных — это система или сервис, из которого поступают данные (например, база данных, API, лог-файлы).

Хранилище данных — это система, предназначенная для длительного хранения данных (например, базы данных, Data Lake).

Вычислительный слой — уровень системы, на котором выполняется обработка данных: очистка, преобразование, агрегация.

Аналитический слой — уровень, на котором выполняется анализ данных, построение моделей машинного обучения и извлечение знаний.

Offline-аналитика — пакетная обработка накопленных данных с задержкой (например, ежедневные расчёты).

Online-аналитика — обработка данных в реальном времени или с минимальной задержкой.

Оркестрация — управление последовательностью выполнения задач, их зависимостями и расписанием.

Data Engineer — специалист, отвечающий за сбор, хранение, обработку данных и построение пайплайна.

Data Scientist — специалист, отвечающий за анализ данных, построение моделей и интерпретацию результатов.

```
In [40]: task_analysis = pd.DataFrame({
        'Параметр': [
            'Предметная область',
            'Основная цель системы',
            'Поставщики данных',
            'Пользователи результата',
            'Какое решение поддерживает система',
            'Требуемая частота обновления результата',
```

```

        'Нужна ли обработка в реальном времени'
    ],
    'Описание': [
        'Система технической поддержки (Service Desk) ИТ-компании',
        'Автоматизация анализа и маршрутизации заявок',
        'Пользователи, email, чат-боты, системы мониторинга',
        'Сотрудники техподдержки, менеджеры, аналитики',
        'Автоматическое назначение заявок и определение приоритета',
        'Почти в реальном времени (несколько секунд/минут)',
        'Да, требуется оперативная обработка'
    ]
}
task_analysis

```

Out[40]:

	Параметр	Описание
0	Предметная область	Система технической поддержки (Service Desk) И...
1	Основная цель системы	Автоматизация анализа и маршрутизации заявок
2	Поставщики данных	Пользователи, email, чат-боты, системы мониторинг...
3	Пользователи результата	Сотрудники техподдержки, менеджеры, аналитики
4	Какое решение поддерживает система	Автоматическое назначение заявок и определение...
5	Требуемая частота обновления результата	Почти в реальном времени (несколько секунд/минут)
6	Нужна ли обработка в реальном времени	Да, требуется оперативная обработка

3. Анализ прикладной задачи

Сделайте краткий вывод по заполненной таблице: кто заинтересован в системе, зачем она нужна, в каком режиме должен формироваться результат.

В системе заинтересованы сотрудники технической поддержки, менеджеры и аналитики, так как она позволяет ускорить обработку заявок и повысить качество обслуживания пользователей. Поставщиками данных являются пользователи, автоматические системы мониторинга, а также каналы коммуникации (email, чат-боты).

Система необходима для автоматического анализа входящих заявок, их классификации и маршрутизации в соответствующие группы поддержки. Это позволяет снизить нагрузку на сотрудников и минимизировать ошибки ручной обработки.

Результат должен формироваться практически в реальном времени, поскольку задержки в обработке заявок напрямую влияют на SLA и удовлетворённость пользователей. Поэтому система должна поддерживать оперативную (online) обработку данных.

In [41]:

```

sources_df[:] = [
    [1, 'Система управления тикетами (Service Desk)', 'структурированные', 'SQL/таблицы', 'постоянно', 'средний'],
    [2, 'Email обращения пользователей', 'неструктурированные', 'текст (email)', 'постоянно', 'средний'],
    [3, 'Чат-бот поддержки', 'полуструктурированные', 'JSON', 'поток', 'средний', 'неполные или неактуальные'],
    [4, 'Логи ИТ-систем и мониторинга', 'поточные', 'log/JSON', 'поток', 'большой', 'шум, высокая нагрузка'],
    [5, 'Справочник сотрудников и команд', 'структурированные', 'CSV/таблицы', 'редко (обновления)', 'средний'],
    [6, 'История заявок (архив)', 'структурированные', 'SQL/хранилище', 'пакетно', 'большой', 'неактуальные']
]

sources_df

```

Out[41]:

№		Источник данных	Тип данных	Формат	Частота поступления	Примерный объём	Проблемы ка
0	1	Система управления тикетами (Service Desk)	структурированные	SQL/таблицы	постоянно	средний	дубликаты, с
1	2	Email обращения пользователей	неструктурированные	текст (email)	постоянно	средний	неструктуриро
2	3	Чат-бот поддержки	полуструктурированные	JSON	поток	средний	непольнекорр
3	4	Логи ИТ-систем и мониторинга	поточковые	log/JSON	поток	большой	шум, высокая
4	5	Справочник сотрудников и команд	структурированные	CSV/таблицы	редко (обновления)	малый	устаревшие ,
5	6	История заявок (архив)	структурированные	SQL/хранилище	пакетно	большой	несогласова пр



Основными источниками данных являются система управления тикетами, email-обращения и чат-бот, так как именно они формируют основной поток заявок пользователей.

Вспомогательными источниками являются справочники сотрудников и история заявок, которые используются для маршрутизации и аналитики.

Наиболее проблемными источниками являются email и логи, так как они содержат неструктурированные или шумные данные. Также потоковые данные из логов требуют обработки в реальном времени и могут создавать высокую нагрузку на систему.

4. Источники данных

Кратко поясните, какие источники являются основными, какие вспомогательными, какие источники наиболее проблемны с точки зрения качества и скорости поступления.

In [42]:

```
data_types_df[:] = [
    ['Структурированные', 'тикеты, справочники сотрудников, история заявок', 'имеют фиксированную'],
    ['Неструктурированные', 'email обращения пользователей', 'содержат текст без фиксированной ст'],
    ['Полуструктурированные', 'данные чат-бота (JSON)', 'имеют структуру, но могут различаться по'],
    ['Потоковые', 'логи систем, события чат-бота', 'поступают непрерывно в реальном времени'],
    ['Архивные', 'история заявок', 'используются для аналитики и обучения моделей'],
    ['Справочные', 'данные о сотрудниках и командах', 'используются для маршрутизации и редко обн'],
]

data_types_df
```

Out[42]:

	Категория данных	Какие данные относятся к этой категории		Почему
0	Структурированные	тикеты, справочники сотрудников, история заявок	имеют фиксированную табличную структуру	
1	Неструктурированные	email обращения пользователей	содержат текст без фиксированной структуры	
2	Полуструктурированные	данные чат-бота (JSON)	имеют структуру, но могут различаться по полям	
3	Потоковые	логи систем, события чат-бота	поступают непрерывно в реальном времени	
4	Архивные	история заявок	используются для аналитики и обучения моделей	
5	Справочные	данные о сотрудниках и командах	используются для маршрутизации и редко обновля...	

5. Классификация данных

Сделайте вывод о разнообразии данных в системе и о том, как это влияет на выбор архитектурных решений.

Система работает с разнородными данными: структурированными (тикеты, справочники), неструктурированными (тексты email) и потоковыми (логи и события).

Это требует использования гибкой архитектуры, которая поддерживает как пакетную, так и потоковую обработку данных. Наличие неструктурированных данных обуславливает необходимость применения методов обработки естественного языка (NLP), а потоковые данные требуют реализации online-компонента системы.

Таким образом, архитектура должна включать инструменты для обработки потоков данных, хранения больших объёмов информации и выполнения аналитических вычислений.

In [43]:

```
pipeline_df['Что происходит на этапе'] = [  
    'Сбор заявок из системы тикетов, email, чат-ботов и логов',  
    'Передача данных через API, стриминг или batch-загрузку в систему',  
    'Удаление дубликатов, фильтрация ошибок, проверка корректности данных',  
    'Нормализация структуры данных, обработка текста (NLP), приведение форматов',  
    'Объединение данных из разных источников в единый набор',  
    'Сохранение данных в централизованное хранилище (Data Lake/БД)',  
    'Формирование признаков (features) для анализа и моделей',  
    'Применение алгоритмов анализа и моделей ML для классификации заявок',  
    'Определение категории заявки, приоритета и ответственной группы',  
    'Передача результатов в систему Service Desk или BI-интерфейс'  
> ]  
  
pipeline_df['Входные данные'] = [  
    'данные из источников (тикеты, email, чат, логи)',  
    'сырые данные',  
    'сырые данные',  
    'очищенные данные',  
    'данные из разных источников',  
    'интегрированные данные',  
    'данные из хранилища',  
    'подготовленные признаки',  
    'результаты модели',  
    'готовые результаты'
```

```

]

pipeline_df['Результат этапа'] = [
    'сырые данные',
    'загруженные данные в систему',
    'очищенные данные',
    'нормализованные данные',
    'единый набор данных',
    'данные, сохранённые в хранилище',
    'набор признаков (features)',
    'классифицированные заявки',
    'сформированные решения (категория, приоритет)',
    'результат, доступный пользователям или системе'
]

pipeline_df

```

Out[43]:

	Этап	Что происходит на этапе	Входные данные	Результат этапа
0	Получение данных	Сбор заявок из системы тикетов, email, чат-бот...	данные из источников (тикеты, email, чат, логи)	сырые данные
1	Загрузка в систему	Передача данных через API, стриминг или batch-...	сырые данные	загруженные данные в систему
2	Очистка данных	Удаление дубликатов, фильтрация ошибок, провер...	сырые данные	очищенные данные
3	Преобразование данных	Нормализация структуры данных, обработка текст...	очищенные данные	нормализованные данные
4	Интеграция источников	Объединение данных из разных источников в един...	данные из разных источников	единый набор данных
5	Хранение данных	Сохранение данных в централизованное хранилище...	интегрированные данные	данные, сохранённые в хранилище
6	Агрегация и расчёт признаков	Формирование признаков (features) для анализа ...	данные из хранилища	набор признаков (features)
7	Аналитическая обработка	Применение алгоритмов анализа и моделей ML для...	подготовленные признаки	классифицированные заявки
8	Формирование результата	Определение категории заявки, приоритета и отв...	результаты модели	сформированные решения (категория, приоритет)
9	Передача результата пользователю/системе	Передача результатов в систему Service Desk ил...	готовые результаты	результат, доступный пользователям или системе

6. Общий пайплайн обработки данных

Опишите полный маршрут движения данных. Укажите, какие этапы являются критическими и почему.

```
In [44]: layers_df = pd.DataFrame({
    'Архитектурный слой': [
        'Источники данных',
        'Интеграция данных',
        'Хранение данных',
        'Вычислительный слой',
        'Аналитический слой',
        'Слой представления',
        'Оркестрация и контроль'
    ],
    'Назначение': [
        'Формирование и предоставление исходных данных о заявках и событиях',
        'Сбор, передача и потоковая загрузка данных в систему',
        'Централизованное хранение сырых и обработанных данных',
        'Очистка, преобразование и подготовка данных к анализу',
        'Анализ данных и применение моделей машинного обучения',
        'Отображение результатов и интеграция с системой поддержки',
        'Управление пайплайном, контроль выполнения и обработка ошибок'
    ],
    'Какие компоненты или классы технологий могут использоваться': [
        'Service Desk системы, email-серверы, чат-боты, системы мониторинга',
        'API, Kafka, стриминговые платформы, ETL-инструменты',
        'Data Lake, SQL/NoSQL базы данных, файловые хранилища',
        'Python, Spark, ETL-пайплайны',
        'scikit-learn, ML-библиотеки, аналитические движки',
        'BI-системы, dashboards, интерфейс Service Desk',
        'Airflow, планировщики задач, системы мониторинга'
    ]
})

layers_df
```

Out[44]:

	Архитектурный слой	Назначение	Какие компоненты или классы технологий могут использоваться
0	Источники данных	Формирование и предоставление исходных данных ...	Service Desk системы, email-серверы, чат- боты,...
1	Интеграция данных	Сбор, передача и потоковая загрузка данных в с...	API, Kafka, стриминговые платформы, ETL-инстру...
2	Хранение данных	Централизованное хранение сырых и обработанных...	Data Lake, SQL/NoSQL базы данных, файловые хра...
3	Вычислительный слой	Очистка, преобразование и подготовка данных к ...	Python, Spark, ETL-пайплайны
4	Аналитический слой	Анализ данных и применение моделей машинного о...	scikit-learn, ML-библиотеки, аналитические движки
5	Слой представления	Отображение результатов и интеграция с системо...	BI-системы, dashboards, интерфейс Service Desk
6	Оркестрация и контроль	Управление пайплайном, контроль выполнения и о...	Airflow, планировщики задач, системы мониторинга

7. Архитектурные слои системы

Поясните, почему система разбита именно на такие слои, какой слой является наиболее важным и где возможны узкие места.

Система разбита на архитектурные слои для обеспечения модульности, масштабируемости и удобства сопровождения. Такое разделение позволяет изолировать различные этапы обработки

данных и упростить развитие системы при увеличении объёма данных или изменении требований.

Слой источников данных отвечает за генерацию информации из различных каналов (тикеты, email, чат-боты, системы мониторинга). Слой интеграции обеспечивает сбор и передачу данных в систему, включая потоковую обработку. Слой хранения предназначен для централизованного накопления как сырых, так и обработанных данных.

Вычислительный слой выполняет ключевые операции по очистке, преобразованию и подготовке данных, а аналитический слой реализует модели машинного обучения для классификации заявок и принятия решений. Слой представления обеспечивает доступ пользователей к результатам системы, а слой оркестрации управляет выполнением всех процессов и контролирует их корректность.

Наиболее важным является **аналитический слой**, так как именно он определяет качество автоматической маршрутизации заявок. Ошибки на этом уровне напрямую влияют на бизнес-результат системы.

К возможным узким местам относятся:

- слой интеграции данных — из-за высокой нагрузки и необходимости обработки потоков в реальном времени
- вычислительный слой — при обработке больших объёмов неструктурированных данных (например, текстов)
- аналитический слой — из-за возможных ошибок моделей и необходимости их регулярного обновления

Таким образом, разделение на слои позволяет повысить устойчивость системы и упростить её масштабирование, но требует особого внимания к производительности и качеству обработки на критических этапах.

8. Архитектурная схема

Проектируемая система построена по многоуровневой архитектуре обработки данных. Данные поступают из различных источников: системы Service Desk, email-каналов, чат-ботов и систем мониторинга.

На первом этапе они попадают в слой интеграции, где осуществляется их сбор и передача в систему обработки через API и потоковые механизмы. Далее данные сохраняются в централизованном хранилище (Data Lake или база данных), где обеспечивается их доступность для последующей обработки.

На вычислительном слое выполняется очистка, фильтрация и преобразование данных, включая обработку текстовой информации. После этого данные передаются в аналитический слой, где применяются модели машинного обучения для классификации заявок, определения их категории и приоритета.

Результаты анализа поступают в слой представления, где становятся доступны пользователям системы (сотрудникам технической поддержки и менеджерам) через интерфейсы или BI-панели. Параллельно слой оркестрации управляет последовательностью выполнения всех этапов, контролирует зависимости и обеспечивает устойчивость пайплайна.

Таким образом, движение данных осуществляется слева направо: от источников через обработку и анализ к формированию конечного результата, который используется для автоматической

маршрутизации заявок.

```
In [45]: modes_df = pd.DataFrame({
    'Процесс': [
        'Приём заявок (тикеты, чат, email)',
        'Загрузка и интеграция данных',
        'Очистка и преобразование данных',
        'Классификация заявок (ML)',
        'Формирование отчетов и аналитики',
        'Обучение и обновление модели'
    ],
    'Режим': [
        'online',
        'online',
        'online',
        'online',
        'offline',
        'offline'
    ],
    'Почему выбран этот режим': [
        'Требуется мгновенная регистрация обращений пользователей',
        'Данные должны поступать без задержек для оперативной обработки',
        'Необходимо быстро подготовить данные для анализа',
        'Маршрутизация должна выполняться в реальном времени',
        'Отчеты не требуют мгновенного обновления',
        'Обучение модели ресурсоемкое и выполняется периодически'
    ]
})

modes_df
```

```
Out[45]:
```

	Процесс	Режим	Почему выбран этот режим
0	Приём заявок (тикеты, чат, email)	online	Требуется мгновенная регистрация обращений пол...
1	Загрузка и интеграция данных	online	Данные должны поступать без задержек для опера...
2	Очистка и преобразование данных	online	Необходимо быстро подготовить данные для анализа
3	Классификация заявок (ML)	online	Маршрутизация должна выполняться в реальном вр...
4	Формирование отчетов и аналитики	offline	Отчеты не требуют мгновенного обновления
5	Обучение и обновление модели	offline	Обучение модели ресурсоемкое и выполняется пер...

9. Offline- и online-контуры

Укажите, какие процессы могут выполняться пакетно, а какие требуют оперативной обработки. Обоснуйте необходимость или отсутствие online-компонента.

```
In [ ]: orchestration_df = pd.DataFrame({
    'Задача': [
        'Загрузка данных',
        'Очистка данных',
        'Преобразование',
        'Интеграция',
        'Агрегация',
        'Аналитическая обработка',
        'Формирование результата'
    ],
    'От каких задач зависит': [
        '',
        'Загрузка данных',
```

```

        'Очистка данных',
        'Преобразование',
        'Интеграция',
        'Агрегация',
        'Аналитическая обработка'
    ],
    'Тип запуска': [
        'Событийный / потоковый',
        'Потоковый',
        'Потоковый',
        'Потоковый + пакетный',
        'Пакетный (по SLA)',
        'Реальное время (ML inference)',
        'Событийный'
    ],
    'Что делать при ошибке': [
        'Повтор загрузки, логирование',
        'Очистка и повтор обработки',
        'Повтор преобразования',
        'Переотправка в очередь',
        'Пересчёт с контрольной точки',
        'Fallback на базовую модель или очередь',
        'Повтор формирования результата'
    ]
}
})
orchestration_df

```

10. Оркестрация и автоматизация

Поясните, какие задачи должны запускаться автоматически, какие выполняются по расписанию, какие — по событию, и почему ручной запуск нежелателен.

```

In [ ]: roles_df = pd.DataFrame({
    'Роль': [
        'Data Engineer',
        'Data Scientist',
        'Пользователь результата / BI-аналитик / эксперт'
    ],
    'Основные функции в проекте': [
        'Проектирование и поддержка пайплайна данных: сбор данных из источников (email, тикеты)',
        'Разработка моделей классификации и маршрутизации заявок, анализ текстов обращений, подготовка отчётов',
        'Использование результатов системы: просмотр и обработка назначенных заявок, контроль качества'
    ]
})
roles_df

```

11. Распределение ролей

Покажите, какие задачи в проекте относятся к инженерии данных, какие — к аналитике, и кто использует итоговый результат системы.

```

In [ ]: risks_df = pd.DataFrame({
    'Риск': [
        'Потеря или задержка поступления заявок',
        'Низкое качество или шум в текстовых данных (email, описания)',
        'Изменение формата входных данных (schema drift)',
        'Перегрузка системы при пиковом потоке заявок',
    ]
})
risks_df

```

```

        'Ошибки классификации ML-модели',
        'Несогласованность данных между источниками'
    ],
    'На каком этапе возникает': [
        'Загрузка данных',
        'Очистка данных',
        'Интеграция данных',
        'Загрузка и обработка потоков',
        'Аналитическая обработка',
        'Интеграция и хранение данных'
    ],
    'Возможные последствия': [
        'Заявки не доходят до системы, нарушение SLA',
        'Ошибочная классификация и маршрутизация заявок',
        'Сбой пайплайна, остановка обработки',
        'Задержки обработки, рост очередей',
        'Неверное назначение заявок специалистам',
        'Некорректная аналитика и дублирование заявок'
    ],
    'Способ снижения риска': [
        'Использование очередей сообщений и retry-механизмов',
        'Очистка текста, NLP-предобработка, фильтрация шума',
        'Контроль схемы данных (schema registry), версионирование',
        'Масштабирование (autoscaling), распределённая обработка',
        'Регулярное переобучение модели и мониторинг качества',
        'Единые идентификаторы, дедупликация, валидация данных'
    ]
}
})

risks_df

```

12. Риски и ограничения

Опишите ключевые риски системы: проблемы качества данных, задержки поступления, ошибки интеграции, изменение схемы источников, проблемы доступа и другие существенные ограничения.

```

In [ ]: summary_df = pd.DataFrame({
    'Компонент системы': [
        'Источники заявок (email, тикеты, чат-боты)',
        'Слой интеграции данных',
        'Хранилище данных (Data Lake / БД)',
        'Слой очистки и нормализации данных',
        'Система извлечения признаков (NLP обработка)',
        'ML-модель классификации и маршрутизации',
        'API выдачи результата в Service Desk',
        'BI/аналитические дашборды'
    ],
    'Назначение': [
        'Сбор входящих заявок из разных каналов',
        'Приведение данных к единому формату',
        'Долговременное хранение заявок и истории',
        'Очистка, удаление шума, дубликатов и ошибок',
        'Преобразование текста в признаки для анализа',
        'Определение категории, приоритета и маршрута заявки',
        'Передача результата обратно в систему поддержки',
        'Анализ нагрузки, SLA и эффективности обработки заявок'
    ],
    'Тип данных': [
        'Текст, JSON, структурированные записи',
        'Полуструктурированные данные',
        'Структурированные и архивные данные',
        'Очищенные структурированные данные'
    ]
})

```

```

        'Текстовые эмбединги, признаки',
        'Модели и предсказания',
        'JSON/API ответы',
        'Агрегированные аналитические данные'
    ],
    'Режим работы': [
        'Потоковый',
        'Потоковый / событийный',
        'Пакетный + потоковый',
        'Потоковый',
        'Потоковый',
        'Online (реальное время)',
        'Online (событийный)',
        'Пакетный (offline)'
    ],
    'Ответственная роль': [
        'Data Engineer',
        'Data Engineer',
        'Data Engineer',
        'Data Engineer',
        'Data Scientist',
        'Data Scientist',
        'Data Engineer',
        'BI-аналитик / Data Analyst'
    ]
})

summary_df

```

13. Итоговая сводная таблица архитектуры

Обобщите систему в виде сводной таблицы: из каких ключевых компонентов она состоит, зачем нужен каждый из них, какие данные он обрабатывает и кто за него отвечает.

```

In [ ]: final_architecture_df = pd.DataFrame({
    'Компонент архитектуры': [
        'Источники данных',
        'Слой приема и интеграции',
        'Слой хранения данных',
        'Слой обработки и очистки',
        'Слой извлечения признаков',
        'ML-модель классификации и маршрутизации',
        'API и сервис выдачи результата',
        'Система мониторинга и аналитики'
    ],
    'Назначение': [
        'Сбор заявок из различных каналов (email, тикеты, чат-боты, мониторинг)',
        'Объединение данных из разных источников в единый формат',
        'Хранение текущих и исторических заявок для обработки и анализа',
        'Очистка данных, удаление дубликатов, нормализация текста',
        'Преобразование текста заявок в признаки для машинного обучения',
        'Автоматическое определение категории, приоритета и маршрута заявки',
        'Передача результата обратно в систему Service Desk для обработки',
        'Контроль качества обработки заявок, SLA и загрузки системы'
    ],
    'Тип данных': [
        'Текст, JSON, структурированные события',
        'Полуструктурированные данные',
        'Структурированные и архивные данные',
        'Очищенные текстовые и структурированные данные',
        'Векторные представления текста, признаки',
        'Модельные предсказания (классы, приоритеты)',
        'JSON-ответы API'
    ]
})

```

```

        'Агрегированные метрики и логи'
    ],
    'Ответственная роль': [
        'Data Engineer',
        'Data Engineer',
        'Data Engineer',
        'Data Engineer',
        'Data Scientist',
        'Data Scientist',
        'Data Engineer',
        'BI-аналитик / Data Analyst'
    ]
})

final_architecture_df

```

14. Итоговые выводы

В выводах отразите:

1. Какую систему вы спроектировали.
2. Какие источники данных являются ключевыми.
3. Каков общий маршрут движения данных.
4. Какие слои системы наиболее важны.
5. Где требуются автоматизация и контроль.
6. Какие риски наиболее существенны.
7. Что оказалось самым сложным при проектировании.

В ходе выполнения работы была спроектирована архитектура Big Data-системы для анализа заявок в службу технической поддержки ИТ-компании с автоматической маршрутизацией обращений. Система предназначена для автоматизации обработки входящих обращений, их классификации по типу и приоритету, а также направления в соответствующие группы поддержки.

Ключевыми источниками данных являются обращения пользователей, поступающие через email, чат-боты, систему Service Desk, а также логи и события мониторинга инфраструктуры. Дополнительную роль играют справочники сотрудников и исторические данные заявок, используемые для маршрутизации и обучения моделей.

Общий маршрут движения данных включает сбор заявок из различных каналов, их интеграцию и приведение к единому формату, очистку и нормализацию, хранение в централизованном хранилище, извлечение признаков, применение модели машинного обучения для классификации и маршрутизации, а также передачу результата в систему Service Desk для дальнейшей обработки.

Наиболее важными слоями системы являются слой интеграции данных, вычислительный слой и аналитический слой, так как именно они обеспечивают корректную обработку данных и формирование интеллектуальных решений на основе заявок пользователей.

Автоматизация особенно критична на этапах загрузки данных, обработки потоков, применения модели и формирования результата, поскольку система работает в режиме близком к реальному времени. Оркестрация необходима для обеспечения последовательности выполнения задач и устойчивости пайплайна при сбоях.

Наиболее существенными рисками являются низкое качество входных данных (особенно текстовых обращений), задержки поступления событий, ошибки интеграции между источниками, а также снижение качества работы модели машинного обучения при изменении характера заявок.

Наиболее сложным этапом проектирования оказалось согласование разнородных источников данных и определение единого пайплайна обработки, который одновременно поддерживает потоковую и пакетную обработку данных без потери качества и производительности.

15. Контрольные вопросы

1. Почему отдельная аналитическая модель не является промышленной системой сама по себе?

Ответ. Аналитическая модель решает только задачу предсказания или классификации, но не обеспечивает полный жизненный цикл данных. Промышленная система включает сбор, хранение, обработку, интеграцию, автоматизацию и мониторинг, поэтому модель является лишь одним из её компонентов.

2. Что такое data pipeline и какие этапы он обычно включает?

Ответ. Data pipeline — это последовательность этапов обработки данных от источников до конечного результата. Обычно он включает сбор данных, загрузку, очистку, преобразование, интеграцию, хранение, анализ и передачу результата пользователю или системе.

3. Чем offline-аналитика отличается от online-аналитики?

Ответ. Offline-аналитика выполняется пакетно на накопленных данных с задержкой (например, ежедневно), а online-аналитика работает в реальном времени или с минимальной задержкой, обрабатывая данные сразу после их поступления.

4. Зачем нужна оркестрация задач обработки данных?

Ответ. Оркестрация нужна для управления порядком выполнения задач, контроля зависимостей, автоматического запуска процессов по расписанию или событию, а также для обработки ошибок и повторного запуска этапов пайплайна.

5. В чём состоит различие между ролями Data Engineer и Data Scientist?

Ответ. Data Engineer отвечает за сбор, хранение, интеграцию и поддержку работы пайплайна данных. Data Scientist занимается анализом данных, построением моделей машинного обучения, извлечением признаков и интерпретацией результатов.

16. Чек-лист готовности

- ☒ Указан номер варианта
- ☒ Описана предметная задача
- ☒ Заполнена таблица анализа задачи
- ☒ Заполнена таблица источников данных
- ☒ Выполнена классификация данных
- ☒ Спроектирован общий пайплайн
- ☒ Описаны архитектурные слои
- ☒ Добавлена архитектурная схема
- ☒ Выделены offline- и online-контуры
- ☒ Описана оркестрация
- ☒ Разграничены роли участников
- ☒ Выявлены риски
- ☒ Заполнена итоговая сводная таблица
- ☒ Сформулированы выводы
- ☒ Даны ответы на контрольные вопросы